

- OG Follow - Image Comparison System Documentation
 - Overview
 - System Architecture
 - Core Components
 - Checkpoints and Models
 - SAM Model Configuration
 - CLIP Model Configuration
 - Processing Logic
 - 1. Image Loading Phase
 - 2. Object Extraction Phase
 - 3. Embedding Generation Phase
 - 4. Similarity Calculation Phase
 - Input Format
 - CSV Structure
 - Expected Data Types
 - URL Requirements
 - Output Format
 - CSV Structure
 - Status Values
 - Console Output Format
 - Performance Metrics
 - Timing Breakdown
 - Memory Usage
 - Test Scenarios
 - Test Case 1: Identical Images
 - Test Case 2: Similar Objects (Different Angles)
 - Test Case 3: Different Objects
 - Test Case 4: Missing URLs
 - Test Case 5: Invalid URLs
 - Test Case 6: Large Images
 - Performance Test Cases
 - Batch Processing Test
 - Memory Stress Test
 - Network Timeout Test
 - Configuration Parameters
 - Threshold Tuning
 - Device Configuration

- [Size Limits](#)
- [Error Handling](#)
 - [Common Exceptions](#)
 - [Recovery Strategies](#)
- [Dependencies](#)
 - [Required Packages](#)
 - [Model Downloads](#)
- [Best Practices](#)
 - [Performance Optimization](#)
 - [Quality Assurance](#)
 - [Security Considerations](#)
- [Troubleshooting](#)
 - [Common Issues](#)
 - [SAM Model Loading Failed](#)
 - [CUDA Out of Memory](#)
 - [Network Timeouts](#)
 - [Low Similarity Scores](#)
 - [Debug Mode](#)
- [Future Enhancements](#)
 - [Planned Features](#)
 - [Performance Improvements](#)

OG Follow - Image Comparison System Documentation

Overview

The `og_follow.py` script is an automated image comparison system that uses advanced AI models to determine similarity between pairs of images. It combines SAM (Segment Anything Model) for object extraction and CLIP for semantic similarity analysis.

System Architecture

Core Components

1. **SAM (Segment Anything Model)** - Object extraction and segmentation
2. **CLIP (Contrastive Language-Image Pre-training)** - Image embedding generation
3. **CSV Processing Engine** - Batch processing of image pairs
4. **Similarity Calculator** - Cosine similarity computation

Checkpoints and Models

SAM Model Configuration

```
SAM_CHECKPOINT = "utils/sam_vit_h_4b8939.pth"  
MODEL_TYPE = "vit_h"
```

Details:

- **Model:** SAM ViT-H (Vision Transformer - Huge)
- **Checkpoint File:** `sam_vit_h_4b8939.pth` (~2.5GB)
- **Purpose:** Automatic mask generation for object extraction
- **Parameters:**
 - `points_per_side=16`: Grid points for mask generation
 - `pred_iou_thresh=0.88`: Quality threshold for predicted masks
 - `stability_score_thresh=0.92`: Stability threshold for mask selection
 - `crop_n_layers=0`: No cropping layers for full image processing

CLIP Model Configuration

```
MODEL_NAME = "openai/clip-vit-base-patch32"
```

Details:

- **Model:** CLIP ViT-Base/32
- **Purpose:** Image embedding generation

- **Input:** 224x224 pixel images
- **Output:** 512-dimensional embedding vectors

Processing Logic

1. Image Loading Phase

- Downloads images from URLs with 20-second timeout
- Converts to RGB format
- Returns numpy array representation

2. Object Extraction Phase

```
def extract_main_object(image, max_size=1024)
```

- Resizes images larger than 1024px (maintains aspect ratio)
- Generates multiple masks using SAM
- Selects largest mask by area (assumes main object)
- Applies mask to isolate main object
- Sets background to black (0 values)

3. Embedding Generation Phase

```
def get_embedding(image)
```

- Converts numpy array to PIL Image
- Processes through CLIP processor
- Generates normalized embedding vectors
- Uses torch.no_grad() for memory efficiency

4. Similarity Calculation Phase

```
similarity = torch.cosine_similarity(emb1, emb2).item()
result = "MATCH" if similarity >= THRESHOLD else "NOT MATCH"
```

- Computes cosine similarity between embeddings
- Applies threshold comparison (0.85)
- Returns similarity score and match status

Input Format

CSV Structure

File: `extracted_instruction_image_only.csv`

Column Name	Description	Required
Iris ID	Unique order identifier	Yes
Complete Toggle Data - imageUrIP1	First image URL (our image)	Yes
Edit Instruction (Input)	Second image URL (customer image)	Yes

Expected Data Types

- **Iris ID:** String/Integer
- **Image URLs:** Valid HTTP/HTTPS URLs
- **Missing Values:** Empty cells or NaN values

URL Requirements

- Must be publicly accessible
- Support common image formats (JPEG, PNG, GIF, BMP)
- Response time < 20 seconds
- File size < 50MB (recommended)

Output Format

CSV Structure

File: `og_image_comparison_output.csv`

Column Name	Description	Format
Order ID	Original Iris ID	String
our using image	First image URL	String
customer image	Second image URL	String
status	Processing result	String
time	Processing duration	String (e.g., "2.45s")

Status Values

- **MATCH**: Similarity ≥ 0.85
- **NOT MATCH**: Similarity < 0.85
- **MISSING URL**: One or both URLs are empty
- **ERROR**: Processing exception occurred

Console Output Format

```
=====
[1/100] Processing: ORDER123
[URL1] https://example.com/image1.jpg
[URL2] https://example.com/image2.jpg
[START] Processing started at 2026-05-06 15:30:00
[TIME] Image loading: 1.23s
[TIME] CLIP embedding: 2.45s
[TIME] Similarity computation: 0.01s
[END] Processing finished at 2026-05-06 15:30:03
[TOTAL] Elapsed time: 3.69s
[RESULT] MATCH
[SIMILARITY] 0.9234
[TIME] 3.69s
```

Performance Metrics

Timing Breakdown

- **Image Loading:** Network-dependent (0.5-5s typical)
- **Object Extraction (SAM):** 1-3s per image
- **Embedding Generation (CLIP):** 0.5-2s per image
- **Similarity Calculation:** <0.01s
- **Total per Pair:** 3-10s typical

Memory Usage

- **SAM Model:** ~2.5GB VRAM/RAM
- **CLIP Model:** ~500MB VRAM/RAM
- **Processing:** ~1GB additional for images

Test Scenarios

Test Case 1: Identical Images

Input:

- URL1: Same image as URL2
- Expected: MATCH with similarity ≈ 1.0

Test Data:

```
Iris ID,Complete Toggle Data - imageUrlP1,Edit Instruction (Input)
TEST001,https://example.com/test.jpg,https://example.com/test.jpg
```

Test Case 2: Similar Objects (Different Angles)

Input:

- Same object, different拍摄角度
- Expected: MATCH with similarity 0.85-0.95

Test Case 3: Different Objects

Input:

- Completely different objects
- Expected: NOT MATCH with similarity < 0.7

Test Case 4: Missing URLs

Input:

- Empty or NaN URL values
- Expected: MISSING URL status

Test Case 5: Invalid URLs

Input:

- Broken or inaccessible URLs
- Expected: ERROR status with exception details

Test Case 6: Large Images

Input:

- High-resolution images (>1024px)
- Expected: Automatic resizing, successful processing

Performance Test Cases

Batch Processing Test

```
# Test with 100 image pairs
python og_follow.py
```

```
# Expected: Complete processing with timing logs
```

Memory Stress Test

```
# Monitor memory usage during processing  
# Expected: Stable memory usage without leaks
```

Network Timeout Test

- Simulate slow network (>20s)
- Expected: Timeout exception handling

Configuration Parameters

Threshold Tuning

```
THRESHOLD = 0.85 # Adjust based on use case
```

- **0.90+**: Very strict matching
- **0.80-0.90**: Standard matching (current)
- **0.70-0.80**: Permissive matching
- **<0.70**: Very permissive

Device Configuration

```
DEVICE = "cpu" # Options: "cpu", "cuda", "mps"
```

Size Limits

```
max_size = 1024 # Maximum image dimension
```

Error Handling

Common Exceptions

1. **Network Errors:** URL timeout, 404, connection refused
2. **Image Errors:** Corrupted files, unsupported formats
3. **Model Errors:** CUDA out of memory, model loading failures
4. **CSV Errors:** Missing files, malformed data

Recovery Strategies

- Skip problematic rows, continue processing
- Log detailed error messages
- Maintain output consistency

Dependencies

Required Packages

```
torch>=1.9.0  
torchvision>=0.10.0  
transformers>=4.0.0  
segment-anything>=1.0  
Pillow>=8.0.0  
numpy>=1.21.0  
pandas>=1.3.0  
requests>=2.25.0
```

Model Downloads

- SAM checkpoint: Downloaded automatically if missing
- CLIP model: Downloaded from HuggingFace hub

Best Practices

Performance Optimization

1. **Batch Processing:** Process multiple images sequentially
2. **Memory Management:** Use `torch.no_grad()` context
3. **Image Optimization:** Resize large images early
4. **Network Caching:** Consider URL caching for repeated processing

Quality Assurance

1. **Input Validation:** Verify URL accessibility
2. **Output Verification:** Check CSV structure integrity
3. **Logging:** Maintain detailed processing logs
4. **Monitoring:** Track success/failure rates

Security Considerations

1. **URL Validation:** Sanitize input URLs
2. **Resource Limits:** Implement processing timeouts
3. **File Size Limits:** Prevent memory exhaustion
4. **Error Disclosure:** Avoid exposing sensitive error details

Troubleshooting

Common Issues

SAM Model Loading Failed

Symptoms: "File not found" or checkpoint errors **Solutions:**

- Verify SAM checkpoint file exists at `utils/sam_vit_h_4b8939.pth`
- Check file integrity (2.5GB size)
- Re-download checkpoint if corrupted

CUDA Out of Memory

Symptoms: GPU memory errors during processing **Solutions:**

- Switch to CPU processing: `DEVICE = "cpu"`
- Reduce batch size or image dimensions
- Close other GPU applications

Network Timeouts

Symptoms: "Timeout exceeded" errors **Solutions:**

- Increase timeout value in `load_image()`
- Check network connectivity
- Verify URL accessibility

Low Similarity Scores

Symptoms: All results show "NOT MATCH" **Solutions:**

- Verify image quality and relevance
- Adjust threshold value
- Check SAM mask generation quality

Debug Mode

Enable detailed logging by modifying log levels:

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

Future Enhancements

Planned Features

1. **GPU Acceleration:** CUDA support for faster processing
2. **Batch Embedding:** Process multiple images simultaneously
3. **Advanced Masking:** Multiple object detection
4. **Similarity Tuning:** Adaptive threshold based on image type
5. **API Integration:** REST API for real-time processing

Performance Improvements

1. **Model Quantization:** Reduce memory usage
 2. **Image Caching:** Store processed embeddings
 3. **Parallel Processing:** Multi-threading for image loading
 4. **Progressive Loading:** Stream large images
-

Version: 1.0

Last Updated: 2026-05-06

Author: OG Diagnostics Team